System Architecture

Agile Software Engineering Capstone

Document field	Team entry
Project title	'Royal Budget' – A Personal Finance & Budgeting Application
Team name	Royal Development
Team members	Jake Hall, Cesar Giner, Jacob Melookaran, Joshua Webber
Student document owner	Cesar Giner
Instructor	Shah, Syed Jawad Hussain
Sponsor or client	Commerce Bank
Version and date	1.0, November 3, 2026
Approval status	In review

Revision history

Version	Date	Author	Summary of change	Reviewer or approval
1.0	November 3, 2026	Cesar Giner	Initial architecture document, reflecting the implemented midterm baseline and the planned features.	Team review

Document outline

Section	Purpose
1 Architecture overview	Purpose, scope, status, and a short explanation of the system
2 Architecture drivers	Use cases, quality attributes, constraints, assumptions, and risks
3 Stakeholders and concerns	What different readers need from the architecture
4 Architecture at a glance	Selected architecture style, major elements, and rationale
5 System context view	Users, external systems, system boundary, and interactions
6 Logical view	Containers, components, responsibilities, and dependencies
7 Process and runtime view	Requests, processes, concurrency, state, and failure behavior
8 Development view	Source organization, modules, ownership, and build structure
9 Deployment view	Where software runs and how environments connect
10 Use case view	Scenarios that validate the other architecture views
11 Data architecture	Entities, relationships, ownership, validation, and migrations
12 Interfaces and APIs	Contracts, authentication, validation, errors, and versioning
13 Security and privacy	Trust boundaries, threats, secrets, identity, and data rules
14 Quality attributes	Performance, reliability, accessibility, maintainability, and testability
15 Architecture decisions	Short records of major decisions and consequences

Section	Purpose
16 Traceability and risks	Links to requirements, tests, prototypes, and unresolved concerns
17 Architecture evolution	How the team reviews and updates the architecture
18 Review and submission	Approvals and final completeness check

1 Architecture overview

1.1 Purpose and scope

This document describes the architecture of Royal Budget Application: the React web client, the FastAPI REST API, and the MySQL database, and how they run in local development and in the planned demo deployment. It is written for the RoyalDev team, the Commerce Bank mentors, and the course instructor.

It does not describe the internals of third-party libraries, the AI provider's model, or any real banking or card integration. The application does not use sensitive private user financial data only.

1.2 Architecture status

Item	Team entry
Architecture version	1.0, November 2, 2026
Current status	Partly implemented
Implementation baseline	Baseline after the midterm demo: registration, sign in, 12-hour sessions, sign out, monthly budgets, transactions, budget progress, and dashboard.
Known gaps	Profile and Settings page, AI assistant, unit and automated tests, file logging, OWASP ZAP scan, session inactivity timeout, and the hosting provider are not yet implemented.

1.3 Short architecture narrative

Royal Budget uses a React web client, a FastAPI REST API, and a MySQL database. The browser talks only to the API, through JSON requests under /api, and never connects to the database. In development, the Vite dev server serves the React app and forwards /api requests to the API, so the browser sees a single origin and the session cookie works.

The API validates each request with Pydantic, identifies the user from an HttpOnly session cookie, applies the budget and transaction rules, and limits every query to the signed-in user. SQLAlchemy maps the API's models to six MySQL tables. Calculated values, such as plan targets and projected savings, are computed from stored data in the client (budgetMath.js) and are never stored. The planned AI assistant will call an external LLM service from the API only, so the API key never reaches the browser.

2 Architecture drivers

Architecture drivers are requirements or risks that strongly influence the system structure. Focus on the few items that change architectural choices.

2.1 Key use cases

Use case ID	Use case	Why it affects architecture	Related view or decision
UC-AUTH-01, UC-AUTH-02	Register and sign in	Requires bcrypt password hashing, unique username and email, and server-side sessions	Security, data, and runtime views; ADR-002
UC-AUTH-03, UC-AUTH-04	Stay signed in and sign out	A session must survive page refreshes, expire after 12 hours, and be revocable on the server	Runtime view; ADR-002, ADR-003
UC-BUD-01	Create a monthly budget	One budget per user per month, plan splits set by the server, and the budget saved together with all its allocations	Data and runtime views; ADR-004
UC-BUD-02, UC-DASH-01	Budget progress and dashboard	Plan vs. actual combines budgets, allocations, and transactions; results are calculated, not stored	Logical view; ADR-005
UC-FIN-01, UC-FIN-02	Record and view transactions	User-owned rows, category kind must match the type, queries by user and month	Data view
UC-AI-01	AI budget insights (planned)	Adds an external dependency; the key must stay in the backend and failures must not break the app	Context view; ADR-007

2.2 Quality attributes

Quality ID	Quality scenario	Measure or response	Architecture effect
QA-01	When a signed-in user requests budgets or transactions, the API returns only that user's data, even if the request changes the month or an ID.	Negative tests with a second user return no other user's data	The user ID always comes from the session; every user-owned query filters by it
QA-02	Budget, transaction, and dashboard requests for a user with up to 1,000 transactions on a local machine (NFR-PERF-001).	Timed API test meets the NFR-PERF-001 target	Month-scoped queries, index on (user_id, transaction_date), recent list capped at 20
QA-03	If the API or database stops during use, pages show a clear error, and a budget is saved with all its allocations or not at all (NFR-REL-001).	Fault test: stop the backend or database	One commit per write with rollback; client error messages
QA-04	Before final delivery, an OWASP ZAP scan of the running app (NFR-SEC-002).	0 critical or high findings	Server-side validation, HttpOnly cookie, CORS allow-list

2.3 Constraints assumptions and architecture risks

Type	Statement	Source	Architecture effect	Owner or validation

Type	Statement	Source	Architecture effect	Owner or validation
Constraint	Use only fabricated financial and card data	Sponsor brief	No real banking connection, no payment processing, and no card fields in the schema	Whole team
Constraint	Full-stack app with a database, a front-end framework, and one CSS framework; no CMS	Sponsor brief	React with Tailwind CSS, FastAPI, and MySQL	Whole team
Assumption	A hosting option can run FastAPI and MySQL for the final demo	Team	Configuration stays in environment variables and standard MySQL so the app can move hosts	Cesar Giner
Risk	AI provider cost, outages, or data privacy	INT-002, Q-001	The assistant could fail or exceed the budget	Cesar Giner-spending cap or message limit

3 Stakeholders and concerns

Stakeholder	Main concerns	Architecture information needed	View or evidence
Users (customers)	Usability, privacy, correct numbers	User journeys and visible system behavior	Use case view (section 10) and context view (section 5)
Developers (RoyalDev team)	Clear responsibilities and interfaces	Components, APIs, dependencies, and source structure	Logical view (section 6) and development view (section 8)
Testers	Testable boundaries and failure behavior	Interfaces, quality scenarios, environments, and logs	Runtime (section 7), deployment (sec 9), and quality (sec 14) sections
Sponsor and instructor	Scope, feasibility, risk, and evidence	Context, major decisions, tradeoffs, and status	Overview (1), drivers (sec 2), ADRs (sec 15), traceability (sec 16)
Security architect (Joshua Webber)	Session handling, secrets, logging, and scan results	Trust boundaries, authentication design, and threats	Security and privacy (13)

4 Architecture at a glance

4.1 Selected architecture style

Royal Budget uses client-server communication in a three-tier structure. The React web client handles presentation and simple form checks; the FastAPI API handles authentication, validation, business rules, and data access; and MySQL stores persistent data.

The team chose this style because it matches the sponsor's required parts (front-end, back-end, and database), keeps the database behind the API, and lets front-end and back-end work proceed in parallel against the documented REST endpoints. Inside the API, code is split by responsibility into small modules

(routes, schemas, models, security, and database) instead of full service and repository layers, which keeps the code simple for a semester-sized project.

4.2 Major architecture elements

Element	Responsibility	Important interface	Data owned or used	Team owner
Web client	Renders the Login, Register, Dashboard, Budget, and Transactions pages, checks forms, calculates plan vs. actual, and redirects to sign in	HTTP JSON to /api (fetch)	Temporary UI state only; no tokens in JavaScript	Cesar Giner (UI/UX: Jake Hall)
API	Authenticates, validates, limits queries to the user, applies budget and transaction rules, and returns JSON	10 REST endpoints under /api	All tables, through SQLAlchemy	Cesar Giner, Jacob Melookaran
Database	Stores persistent data and enforces keys, unique rules, and check constraints	SQL from the API only, as budget_app_user(not root)	users, sessions, categories, budgets, budget_allocations, transactions	Jacob Melookaran
AI/LLM service (planned)	Answers budgeting questions from a summary of the user's data	HTTPS JSON, called from the API only	Receives a budget summary and the question; nothing stored by the app	Cesar Giner, Joshua Webber

5 System context view

5.1 Context diagram

Figure 1 shows Royal Budget as one system, the person who uses it, and the planned AI service.



Figure 1. System context (v1.0, November 9, 2026). Dashed elements are planned.

5.2 Context explanation

Actor or external system	Interaction with the system	Information exchanged	Trust or dependency concern
Registered user	Uses a web browser to register, sign in, plan budgets, and record transactions	Credentials, profile, budgets, and fabricated transactions	Must see only their own data; the password is never stored in plain text
AI/LLM service (planned)	The API sends a question with a budget summary and receives a text answer	Budget and transaction summary only; no passwords, tokens, or account details	External dependency: cost, outages, and data leaving the system; key kept in the backend .env

Actor or external system	Interaction with the system	Information exchanged	Trust or dependency concern
Commerce Bank	No runtime interaction; provides requirements and reviews demos	None at runtime	Requires fabricated data only

6 Logical view

6.1 Container diagram

Figure 2 shows the three containers inside the system boundary. In development, the Vite dev server also forwards /api requests to the API, so the browser and API share one origin.

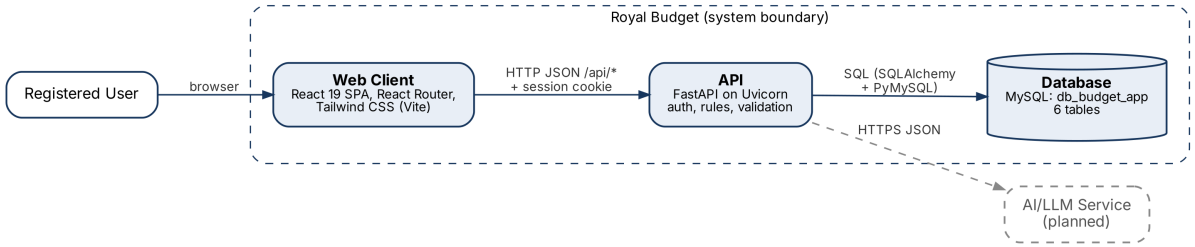


Figure 2. Containers (v1.0, November 9, 2026). Dashed elements are planned.

6.2 Component view

Figure 3 shows the components of the API, the most important container. The routes in main.py handle each endpoint; shared responsibilities live in separate modules.

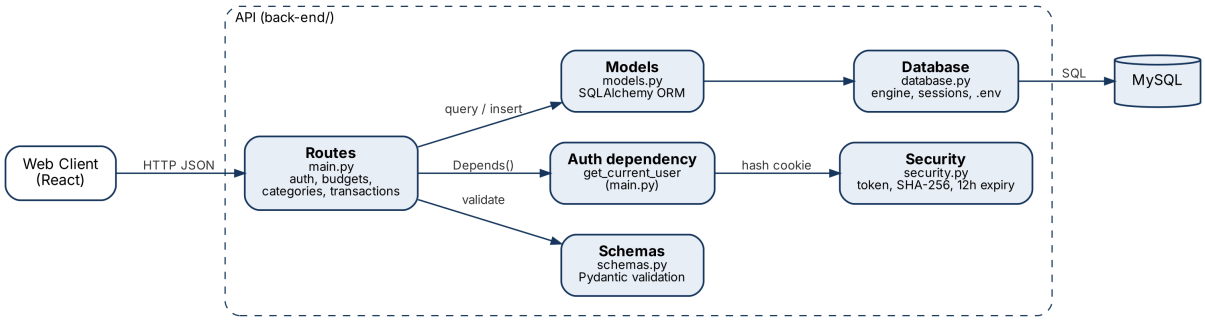


Figure 3. API components (v1.0, November 9, 2026).

Component	Responsibility	Depends on	Provided interface	Requirement or use case

Component	Responsibility	Depends on	Provided interface	Requirement or use case
Routes (main.py)	Defines the 10 endpoints and applies business rules: plan splits, one budget per month, allocations for every need and want category, and category kind matching the transaction type	Schemas, auth dependency, models, get_db	REST endpoints under /api	UC-AUTH-01 to 04, UC-BUD-01, UC-FIN-01, UC-FIN-02
Auth dependency (get_current_user)	Reads the session cookie, finds its hash, rejects missing, revoked, or expired sessions with 401, and returns the valid user	security.py, models, database session	FastAPI Depends() on every protected route	UC-AUTH-03 and all protected use cases
Security (security.py)	Creates random session tokens, hashes them with SHA-256, and sets the 12-hour expiry in UTC	Python secrets and hashlib	new_session_token, hash_token, session_expiry	UC-AUTH-02, UC-AUTH-03
Schemas (schemas.py)	Validates request bodies and shapes responses: lengths, patterns, amount ranges, and month format	Pydantic	Request and response models	UC-AUTH-01, UC-BUD-01, UC-FIN-01
Models (models.py)	Maps the six tables and their relationships to Python classes	SQLAlchemy, database.py	ORM classes	All use cases
Database (database.py)	Loads settings from .env and creates the engine (UTC time zone, connection pre-ping) and SessionLocal	python-dotenv, PyMySQL	get_db: one session per request	All use cases

7 Process and runtime view

7.1 Key request or event flows

Figure 4 shows the two most important flows: signing in, and an authenticated request that creates a budget. Every other protected request starts with the same session check.

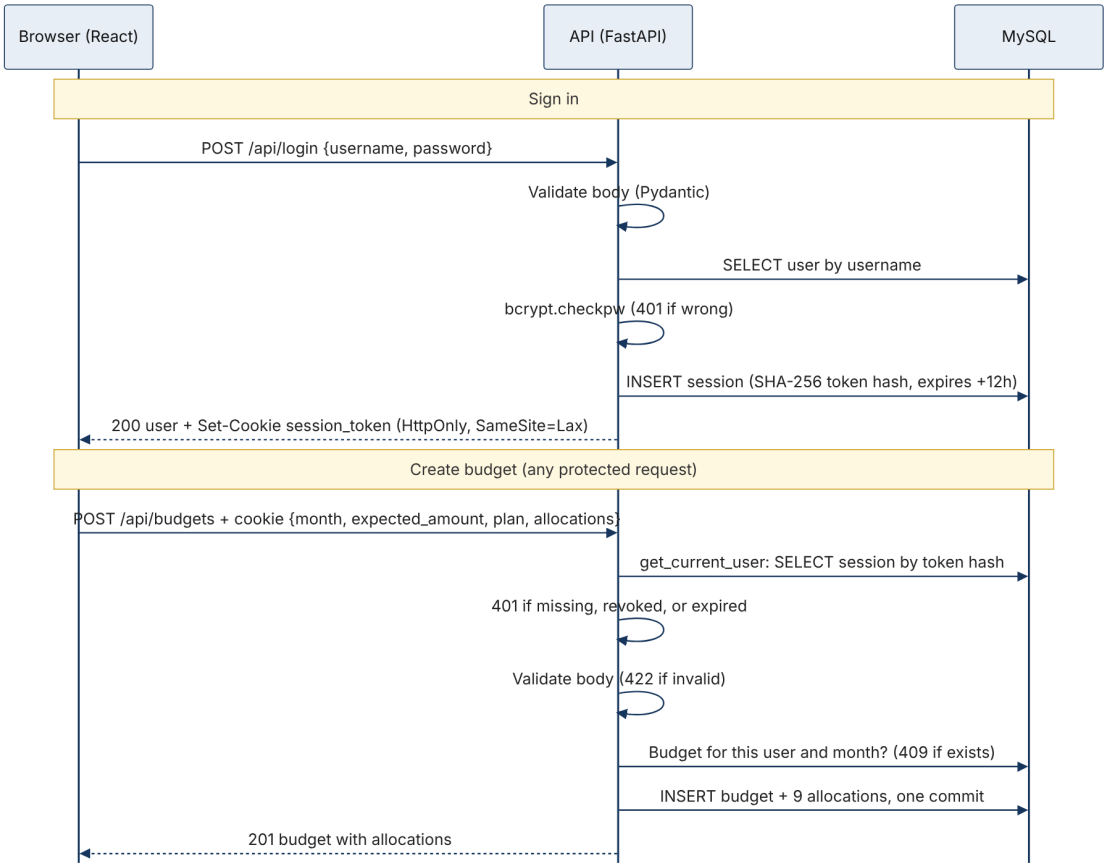


Figure 4. Sign in and create budget (v1.0, November 9, 2026).

Sign out: the client sends POST /api/logout. The API marks the session as revoked and deletes the cookie, so the next request returns 401 and the client redirects to /login.

7.2 Processes concurrency state and failure

Concern	Design decision	Failure behavior	How the team will test or observe it
Session state	Server-side sessions in the sessions table; the browser holds only a random token in an HttpOnly, SameSite=Lax cookie; 12-hour lifetime	A missing, revoked, or expired session returns 401, and the page redirects to /login	Manual test now; automated API test planned
Concurrent update	Unique keys on (user_id, budget_month), username, and email; the API catches IntegrityError	A duplicate budget or account returns 409 instead of a server error	Duplicate request test (planned)
Database failure	Connection pre-ping; each budget is saved in one commit with rollback on error	The page shows "Could not reach the server" or a load error; no partial budget is saved (NFR-REL-001)	Fault test: stop MySQL or the backend during use
External dependency (AI, planned)	Timeout and a per-user message limit on calls to the LLM service	The assistant shows an "unavailable" message; the rest of the app keeps working	Fault test with an invalid key (planned)

8 Development view

8.1 Source and module organization

The repository has one top-level folder per container plus a docs folder. Folder and module names match the names used in the diagrams.

Repository or module	Purpose	Main dependencies	Build or test command	Owner
front-end/	React app: pages, routes, budgetMath.js, Tailwind styles, and the Vite /api proxy	React 19, React Router, Tailwind CSS v4, Vite	npm install; npm run dev; npm run build; npm run lint	Cesar Giner (UI/UX: Jake Hall)
back-end/	FastAPI app: routes, schemas, models, security, and database connection	FastAPI, Uvicorn, SQLAlchemy, Pydantic, bcrypt, PyMySQL, python-dotenv	pip install -r requirements.txt; fastapi dev main.py; pytest (planned)	Cesar Giner, Jacob Melookaran
database/	schema.sql: creates the database, six tables, constraints, indexes, and 14 seed categories	MySQL 9	Run schema.sql in MySQL Workbench (development only; drops all tables)	Jacob Melookaran
docs/	Charter, requirement specifications, use cases, database schema, architectural, and planning documents	None	None	Joshua Webber, Cesar Giner

8.2 Team and integration structure

Cesar Giner leads the project and builds most of the front-end and API. Jacob Melookaran owns the database schema and supports the back-end, Jake Hall owns UI/UX design (Figma), and Joshua Webber owns security standards and documentation. The REST endpoints and their JSON shapes (Section 12) are the shared contract between the front-end and the API.

The team works in one GitHub repository (Cesar-KC/RoyalDevCapstoneProject) and currently commits to main after testing locally. Changes to an endpoint or the schema are discussed with the team first and recorded in the README. Conway's Law risk: because one person writes most of both tiers, knowledge is concentrated; the README, this document, and the requirements and use case documents reduce such risk but does not eliminate. Communication is a priority in team environment.

9 Deployment view

9.1 Deployment diagram

Figure 5 shows the current local development setup and the planned demo deployment. The hosting provider has not been decided.

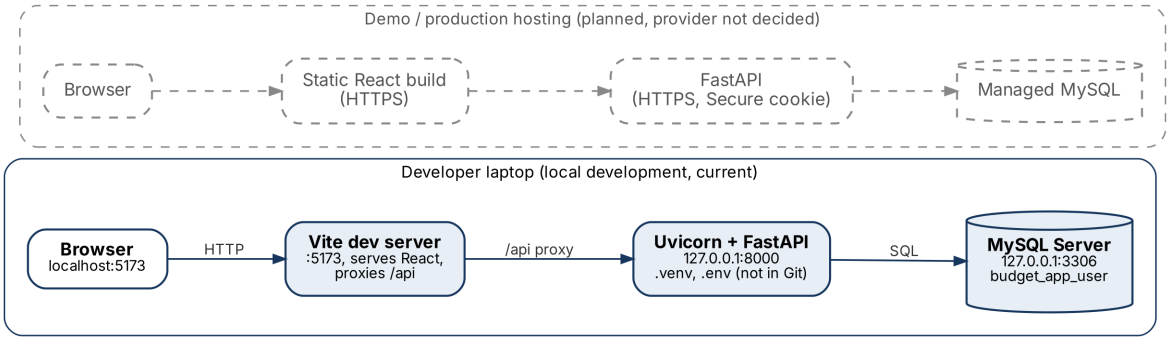


Figure 5. Deployment (v1.0, November 9, 2026). Dashed elements are planned.

9.2 Environment inventory

Environment	Purpose	Deployed elements	Data	Configuration and secrets	Owner
Local development	Build, manual testing, and demos	Vite dev server, FastAPI on Uvicorn, local MySQL	Fabricated developer data;	back-end/.env for database settings, excluded by .gitignore; .env.example in Git	Each developer
Test (planned)	Automated unit and API tests	FastAPI app with a separate test database	Fabricated test fixtures	Test settings through environment variables	Whole team
Demo / production (planned)	Final sponsor demo	Static React build, FastAPI, and managed MySQL over HTTPS	Fabricated demo data	Host's secret settings; Secure cookie on; CORS limited to the site's domain	Whole Team

10 Use case view

The use case view is the plus one view. It shows how the logical, runtime, development, and deployment views work together for an important scenario.

Scenario	Logical elements	Runtime flow	Deployment nodes	Evidence
Register and sign in (UC-AUTH-01, UC-AUTH-02)	RegisterPage, LoginPage, register and login routes, schemas, security, users and sessions tables	Validate, check unique username and email, hash with bcrypt, and store; on sign in, verify the hash, store the token hash, and set the cookie	Browser, Vite, API, MySQL	Figure 4; manual test;
Create a monthly budget (UC-BUD-01)	BudgetPage, POST /api/budgets, auth dependency, schemas, models	Authenticate, validate, check one budget per month, and save the budget with 9 allocations in one commit	Browser, Vite, API, MySQL	Figure 4; DB data, midterm demo
Record and view transactions (UC-FIN-01, UC-FIN-02)	TransactionsPage, transaction routes, categories	Authenticate, check the category kind against the type, and insert; list a month newest first	Browser, Vite, API, MySQL	DB data, Midterm demo

Scenario	Logical elements	Runtime flow	Deployment nodes	Evidence
View dashboard (UC-DASH-01, UC-BUD-02)	DashboardPage, budgetMath.js, five GET endpoints	Load the user, categories, budget, month's transactions, and 5 most recent in parallel, then calculate in the client	Browser, Vite, API, MySQL	Manual View, Midterm demo
Sign out (UC-AUTH-04)	Sign-out button, POST /api/logout	Revoke the session, delete the cookie, and redirect to /login	Browser, Vite, API, MySQL	Manual test

11 Data architecture

11.1 Data model

Figure 6 shows the six tables. Every user-owned table links to users and is deleted with the user (ON DELETE CASCADE). Categories are shared reference data. Schema changes are made in database/schema.sql; a migration tool is not used yet.

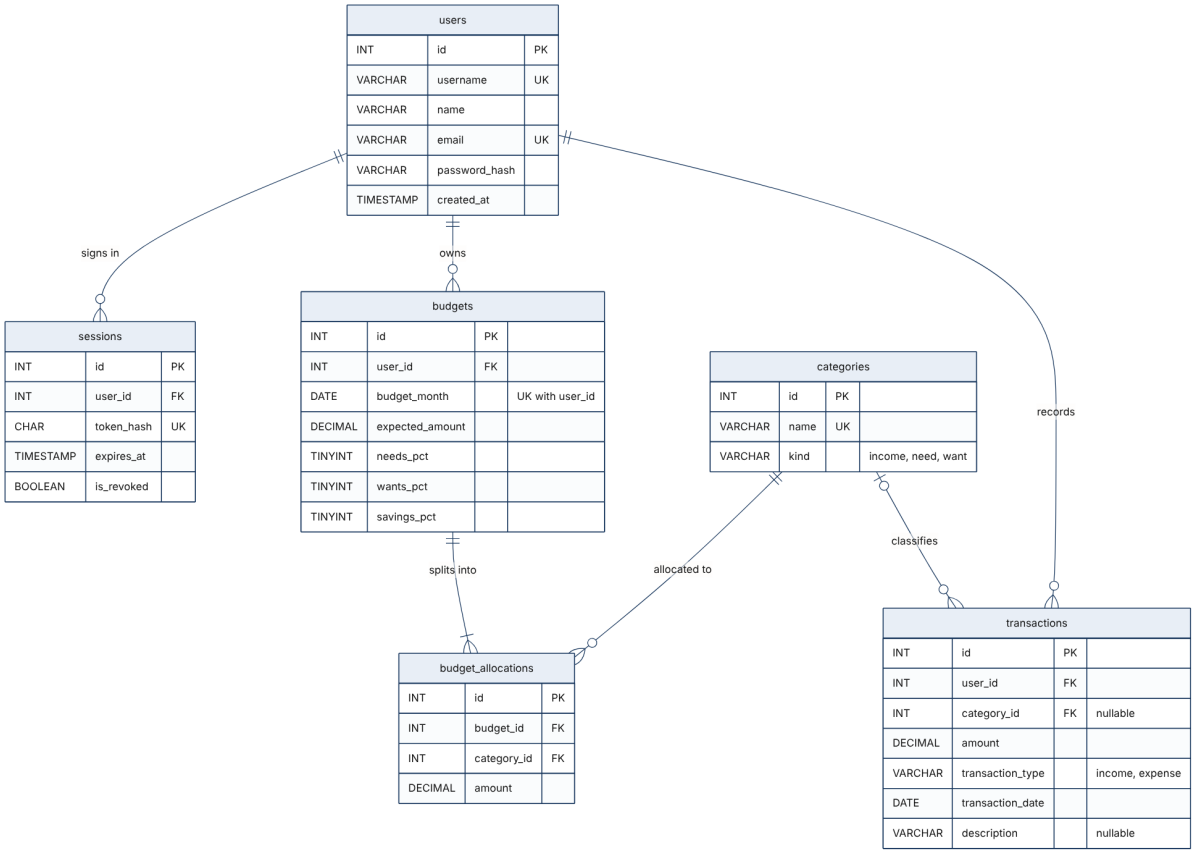


Figure 6. Data model (v1.0, November 9, 2026). PK primary key, FK foreign key, UK unique.

11.2 Data dictionary and ownership

Entity or data set	Purpose	Important fields	Owner or access rule	Validation retention or migration
User (users)	Account identity and profile	id, username, name, email, password_hash, created_at	API only; a user reads only their own account (/api/me)	Unique username (3–50 characters: letters, numbers, underscore) and email; bcrypt hash only
Session (sessions)	Server-side sign-in record	user_id, token_hash, expires_at, is_revoked	API only; never returned to the client	Unique SHA-256 hash; the raw token exists only in the cookie; revoked on sign out; expires after 12 hours
Category (categories)	Shared list: 6 needs, 3 wants, 5 income	name, kind	Read-only for users	kind is income, need, or want; seeded by schema.sql
Budget (budgets)	A user's plan for one month	user_id, budget_month, expected_amount, needs_pct, wants_pct, savings_pct	Queries always filter by the signed-in user	One per user per month; first day of the month; percentages set by the server and sum to 100
Budget allocation (budget_allocations)	Planned amount for each need and want category	budget_id, category_id, amount	Reached only through the user's budget	One row per category per budget; amount at least 0; deleted with the budget
Transaction (transactions)	A recorded income or expense	user_id, category_id, amount, transaction_type, transaction_date, description	Queries always filter by the signed-in user	Amount greater than 0 with up to 2 decimals; category kind matches the type; fabricated data only

12 Interfaces and APIs

12.1 Interface catalog

Interface ID	Provider and consumer	Purpose	Protocol or format	Authentication	Version or status
API-01	API provides; web client consumes	Create an account	POST /api/register, JSON	None	Implemented
API-02	API provides; web client consumes	Sign in and set the session cookie	POST /api/login, JSON	None	Implemented
API-03	API provides; web client consumes	Get the signed-in user	GET /api/me	Session required	Implemented
API-04	API provides; web client consumes	Sign out and clear the cookie	POST /api/logout	Session cookie (if present)	Implemented
API-05	API provides; web client consumes	List categories	GET /api/categories	Session required	Implemented
API-06	API provides; web client consumes	Get a month's budget with allocations	GET /api/budgets?month=YYYY-MM	Session required	Implemented
API-07	API provides; web client consumes	Create a month's budget	POST /api/budgets, JSON	Session required	Implemented
API-08	API provides; web client consumes	List a month's transactions	GET /api/transactions?month=YYYY-MM	Session required	Implemented
API-09	API provides; web client consumes	List the newest transactions (limit 1–20)	GET /api/transactions/recent?limit=N	Session required	Implemented
API-10	API provides; web client consumes	Record an income or expense	POST /api/transactions, JSON	Session required	Implemented
API-11	AI/LLM service provides; API consumes	Answer a budgeting question	HTTPS JSON (provider to be decided)	API key in the backend .env	Planned

12.2 Contract details

Contract field	Team entry
Endpoint or interface	POST /api/budgets (API-07)

Contract field	Team entry
Purpose	Create the signed-in user's budget for one month, with an allocation for every need and want category.
Request example	{"month": "2026-11", "expected_amount": 4000, "plan": "balanced", "allocations": [{"category_id": 1, "amount": 500}, {"category_id": 3, "amount": 1200}]}
Success response	201 Created with {id, month, expected_amount, needs_pct: 50, wants_pct: 30, savings_pct: 20, created_at, allocations}. All 9 need and want categories are returned; categories not sent are saved as 0.
Validation errors	422 with {"detail": ...} for a bad month format, an amount outside 1 to 1,000,000,000, an unknown plan, an income or unknown category, or a repeated category. 409 {"detail": "A budget already exists for this month"}.
Authentication and authorization	Session cookie required; otherwise 401 {"detail": "Not authenticated"}. The user ID comes from the session, never from the request, so a user can only create their own budget.
Compatibility or versioning	The front-end and API live in the same repository and change together. Breaking changes to a request or response are agreed by the team and updated in the README endpoint table. URL versioning (/api/v1) is not used yet.

#All other API contracts follow same philosophy

13 Security and privacy

13.1 Trust boundaries and threats

The main trust boundary is between the browser and the API: everything the browser sends is untrusted until the API validates it and checks the session. A second boundary separates the API from the database, which accepts connections only from the API's restricted user, and a third will separate the API from the planned AI service.

Threat or misuse case	Affected asset	Architecture control	Verification evidence	Owner
A user changes the month or an ID to read another user's data	Budgets and transactions	The user ID always comes from the session; every user-owned query filters by it; no endpoint accepts a user ID	Manual negative test; automated authorization tests (planned)	Cesar Giner
Malicious input reaches a database query	Database integrity	Pydantic validation, SQLAlchemy bound parameters, and database check constraints	ZAP scan; API tests (planned)	Jacob Melookaran
A session token is stolen through a script or a database leak	User sessions	HttpOnly cookie (no JavaScript access); only the SHA-256 hash is stored; 12-hour expiry; revoked on sign out; React escapes rendered text	Code review; ZAP scan	Joshua Webber
Password guessing or a leaked password table	User accounts	Salted bcrypt hashes; the same error for a wrong username or password	Code review; failed sign-in logging (planned)	Joshua Webber

Threat or misuse case	Affected asset	Architecture control	Verification evidence	Owner
Secrets committed to Git	Database password, AI key	.env is in .gitignore; .env.example holds placeholders; the database user has only SELECT, INSERT, UPDATE, DELETE	Repository review	Whole team

13.2 Security and privacy decisions

Area	Team decision
Authentication	Username and password with server-side sessions: a random 32-byte token in an HttpOnly, SameSite=Lax cookie (Secure in production), its SHA-256 hash stored in sessions, a 12-hour lifetime (NIST SP 800-63B AAL2), and revocation on sign out. A 30-minute inactivity timeout is planned.
Password handling	bcrypt with a per-password salt; passwords must be 8 to 72 bytes. Plain-text passwords are never stored or logged.
Authorization	One role: registered user. Every protected route uses get_current_user, and every user-owned query filters by the session's user ID.
Input and output protection	Pydantic checks types, lengths, patterns, and ranges; SQLAlchemy uses bound parameters; MySQL check and unique constraints back up the rules; React escapes rendered text. There are no file uploads.
Secrets	Database settings live in back-end/.env (git-ignored), and the app connects as budget_app_user with limited privileges. The AI key will live only in the backend .env; in production, the host's secret settings are used.
Logging	(Planned) Errors and failed sign-ins are written to a file with the time and username, never passwords, tokens, or cookies.
Financial data	Fabricated data only: no card numbers, bank accounts, or real banking connections. Users enter amounts manually.

14 Quality attributes

Quality attribute	Target or scenario	Architecture tactic	Evidence and owner
Performance	Budget, transaction, and dashboard requests meet NFR-PERF-001 for a user with up to 1,000 transactions on a local machine(currently only 20 rows)	Month-scoped queries, index on (user_id, transaction_date), recent list capped at 20; the dashboard sends its requests in parallel	Timed API test(planned); Cesar Giner
Reliability	If the API or database stops, pages show an error instead of failing silently; a budget is saved with all its allocations or not at all (NFR-REL-001)	One commit per write with rollback, connection pre-ping, and unique constraints that turn duplicates into 409	Fault test(planned); Jacob Melookaran

Quality attribute	Target or scenario	Architecture tactic	Evidence and owner
Maintainability	An endpoint or page can be added without changing unrelated code; backend coverage of at least 70% (NFR-MNT-001)	Separate modules for schemas, models, security, and database; shared budgetMath.js; configuration in .env	Code coverage (planned);Cesar Giner
Testability	Business rules can be tested without a browser	FastAPI dependency overrides for get_db and get_current_user; a test database with fabricated fixtures	pytest with FastAPI TestClient (planned); whole team
Observability	During a failure, the team can tell which request failed and why	Uvicorn request logs in the console now; (file logging of errors and failed sign-ins planned)	Log review during testing and the demo; Joshua Webber

15 Architecture decisions

Create a short Architecture Decision Record for every decision that would be expensive or disruptive to change later. Keep rejected alternatives so future teammates understand the reasoning.

15.1 Decision index

ADR ID	Decision title	Status	Date	Owner	Related requirements
ADR-001	Three-tier architecture: React, FastAPI, and MySQL	Accepted	October 2026	Team	Sponsor brief (3.2), CHG-002
ADR-002	Server-side sessions in an HttpOnly cookie instead of JWT	Accepted	October 2026	Cesar Giner	FR-AUTH-02, FR-AUTH-03, FR-AUTH-04, NFR-SEC-001
ADR-003	12-hour session lifetime with no "remember me"	Accepted	October 30, 2026	Joshua Webber	FR-AUTH-03, CHG-004
ADR-004	The server sets plan splits and saves every need and want allocation	Accepted	October 2026	Cesar Giner	FR-BUD-01, FR-BUD-02, BR-006
ADR-005	Calculate plan vs. actual in the client; store nothing calculated	Accepted	October 2026	Jake Hall, Jacob	FR-BUD-04, FR-DASH-01
ADR-006	Vite dev proxy for /api so the browser and API share one origin	Accepted	October 2026	Team	INT-001
ADR-007	Call the AI service only from the API	Proposed	November 2026	Team	FR-AI-01, INT-002

15.2 Architecture Decision Record template

Field	Content

Field	Content
Decision	ADR-002: Use server-side sessions: a random token in an HttpOnly cookie, with only its SHA-256 hash stored in the sessions table.
Status and date	Accepted, October 2026. The lifetime was changed to 12 hours on October 30, 2026 (ADR-003).
Context	Users must stay signed in across page refreshes (UC-AUTH-03), signing out must really end access (UC-AUTH-04), and the app handles personal financial data. In development, the Vite proxy puts the browser and API on one origin.
Options considered	1. JWT stored in localStorage. 2. JWT in a cookie. 3. Server-side session token in an HttpOnly cookie.
Choice and reason	Option 3. JavaScript cannot read an HttpOnly cookie, so a script injection cannot steal it. The server can revoke a session immediately on sign out, which a JWT cannot do without a deny list. Storing only a hash means a database leak does not expose usable tokens.
Consequences	One extra indexed database lookup per request. Expired and revoked rows build up and need a cleanup job later (AR-06). Production must use HTTPS with the Secure flag and a single site so SameSite=Lax keeps working.
Owner and reviewers	Joshua Webber (security) and the team
Related evidence	security.py, get_current_user in main.py, the sessions table in schema.sql, NIST SP 800-63B, and manual sign-in, refresh, and sign-out tests

#NOTE: Finish ADR for the other decisions made(1,3-7)

16 Traceability and risks

16.1 Architecture traceability

Requirement or quality ID	Architecture element or decision	Verification evidence	Implementation status
FR-AUTH-01, FR-AUTH-02 Register and sign in	Register and login routes, schemas.py, bcrypt, users table, ADR-002	Manual tests; TC-AUTH-01 and 02 (planned)	Implemented
FR-AUTH-03, FR-AUTH-04 Sessions and sign out	get_current_user, security.py, sessions table, ADR-002, ADR-003	Manual refresh and sign-out tests; TC-AUTH-03 and 04 (planned)	Implemented
FR-BUD-01 to FR-BUD-03 Create a budget	POST /api/budgets, PLAN_SPLITS, budgets and budget_allocations tables, ADR-004	Midterm demo; TC-BUD-01 to 03 (planned)	Implemented
FR-BUD-04, FR-DASH-01 Progress and dashboard	budgetMath.js, BudgetPage, DashboardPage, ADR-005	Midterm demo; TC-BUD-04, TC-DASH-01 (planned)	Implemented
FR-FIN-01, FR-FIN-02 Transactions	Transaction routes, transactions table, category kind check	Midterm demo; TC-FIN-01 and 02 (planned)	Implemented
FR-AI-01 AI assistant	AI route (planned), API-11, ADR-007	Usage and fault tests (planned)	Planned

Requirement or quality ID	Architecture element or decision	Verification evidence	Implementation status
FR-PROF-01, FR-PROF-02 Profile and settings	Profile page and endpoints (planned)	TC-PROF-01 and 02 (planned)	Planned
NFR-SEC-001, NFR-SEC-002 Security	bcrypt, session design, file logging, and ZAP scan (Section 13)	ZAP report and log review	Active

16.2 Architecture risks and open questions

ID	Risk or question	Why it matters	Validation or response	Owner	Target date	Status
AR-01	Which hosting option can run FastAPI and MySQL for the final demo?	Deployment could fail late	Partial Deployment?	Team	Sprint 4	Open
AR-02	/api/login has no limit on failed attempts	Password guessing is possible	Add an attempt limit or delay, and log failed sign-ins	Joshua Webber	Sprint 5	Open
AR-03	AI provider cost, privacy, and outages	Unexpected charges! or a broken feature	Spending cap, per-user message limit, summary-only data, and a fallback message	Cesar Giner	Sprint 4	Open
AR-04	No automated tests yet	Regressions and a missed coverage target	pytest with FastAPI TestClient and a test database	Whole team	Sprint 4	Open

17 Architecture evolution

Cesar Giner maintains this document, and the team reviews it once per sprint during sprint planning. Whoever changes a component, endpoint, table, or deployment updates the matching diagram and table in the same sprint. Planned elements are labeled "planned" in the text and drawn dashed in diagrams. A new ADR is added for any decision that would be expensive to reverse, such as the hosting provider or the AI provider.

17.1 Diagram rules

☑ Give each diagram a title, version or date, purpose, and legend when needed.

☑ Label arrows with the interaction or protocol, such as HTTPS JSON request or SQL query.

☑ Use the same component names in diagrams, source folders, interfaces, and tables.

☑ Use one readable diagram for one main question instead of one giant diagram.

☑ Mark proposed and implemented elements clearly and update their status each sprint.

☑ Do not use screenshots of code as architecture diagrams. (LOL)

18 Review and submission

18.1 Review decisions

Reviewer	Role	Decision	Conditions or comments	Date
Commerce Bank mentors	Project sponsor or mentor	Pending		
Shah, Syed Jawad Hussain	Instructor	Pending		
Cesar Giner	Architecture lead	Approved	Updated and current	11/6/2026

18.2 Final submission checklist

☑ Architecture drivers come from real use cases, nonfunctional requirements, constraints, and risks.

☑ The document includes context, logical, runtime, development, deployment, and use case views as needed.

☑ Diagrams use readable labels and names that agree with the implementation.

☑ The data model protects ownership boundaries and prohibits real financial or card data.

☑ Interfaces describe authentication, authorization, validation, success, and error behavior.

☑ Security, secrets, logging, accessibility, reliability, performance, maintainability, and testability are addressed.

☐ Major decisions include alternatives, rationale, consequences, owner, and date. (Full record for ADR-002; the others are in the index.) [Finish ADRs]

☑ Architecture claims link to requirements and verification evidence.

☑ Proposed and implemented elements are clearly distinguished.

☑ All bracketed prompts and unused example rows have been removed.

☐ The version, date, author list, diagram dates, and review decisions are current. (pending.)